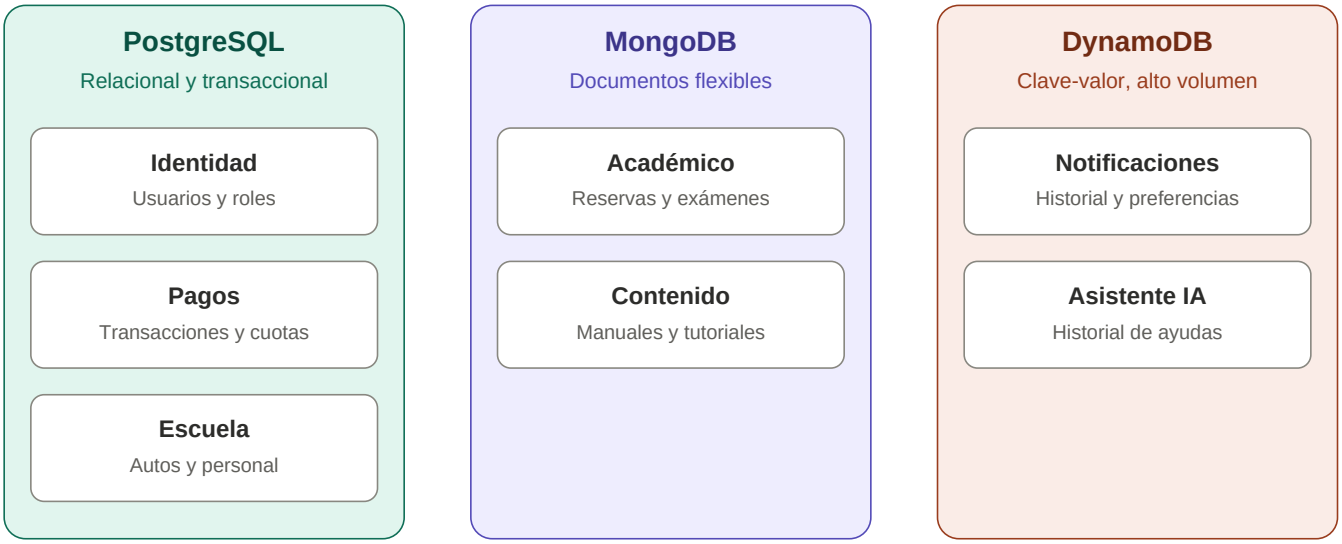
DriveHub: bases de datos y micro-frontends

Continuación de la arquitectura de microservicios. Requisitos: usar PostgreSQL, MongoDB y DynamoDB, y usar micro-frontends con Angular y React.

1. Bases de datos por microservicio



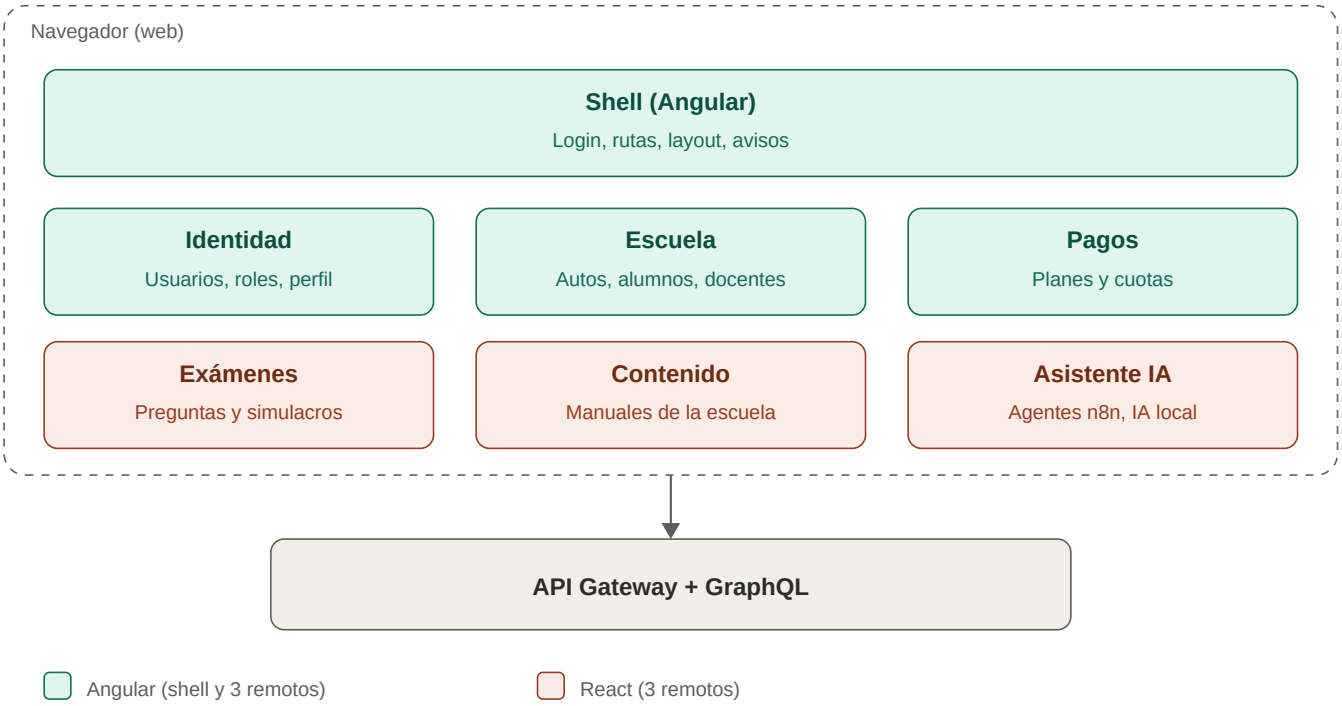
Base	Servicios	Por qué
PostgreSQL	Identidad, Pagos, Escuela	Datos relacionales con integridad fuerte. Pagos necesita transacciones ACID, y la saga de rollback depende de que cada paso se confirme o se revierta bien. Identidad y Escuela son tablas con claves foráneas (usuarios, roles, autos, docentes).
MongoDB	Académico, Contenido	Los exámenes tienen preguntas con estructuras variables (opción múltiple, imagen, verdadero/falso); un documento por simulacro con sus preguntas anidadas lo resuelve bien. Los manuales y tutoriales son metadatos más enlaces a archivos, sin relaciones complejas.
DynamoDB	Notificaciones, Asistente IA	Los dos acumulan mucho historial que se consulta siempre por usuario y fecha (clave de partición `usuarioId`, clave de orden `timestamp`). La expiración automática (TTL) sirve para limpiar notificaciones viejas.

Puntos a tener presentes

- **Doble reserva en Académico:** al estar en MongoDB, se evita que dos estudiantes reserven el mismo horario con un índice único sobre `(instructor, fecha, hora)`. Las operaciones que tocan varios documentos requieren un replica set. Si prefieren no depender de eso, la alternativa es mover Académico a PostgreSQL y dejar MongoDB solo para el banco de preguntas y Contenido.
- **Estado de la saga:** puede guardarse en DynamoDB (un registro por saga con pasos hechos y compensaciones pendientes, actualizado por clave), sin mezclarlo con las tablas de Pagos.
- **DynamoDB en desarrollo:** necesita una cuenta de AWS, pero para desarrollar basta con la imagen `amazon/dynamodb-local` en el `docker-compose`. El free tier alcanza para la demo.

2. Micro-frontends (web)

Solo cubren la web. El móvil (Flutter) lo usan el estudiante y el instructor y queda fuera del esquema de micro-frontends. Se dividieron por dominio, igual que los microservicios, para que cada dueño tenga su porción completa de pantalla y API.



Micro-frontend	Framework	Casos de uso web	Sugerido para
Shell	Angular	Login (CU02), cierre de sesión, menú por rol, campana de avisos	Leonel
Identidad	Angular	CU01, CU05, CU06 (usuarios, roles y perfil)	Mauro
Escuela	Angular	CU07 a CU11 (autos, estudiantes, docentes, asignaciones)	Mauro
Pagos	Angular	CU25 (planes de pago y cuotas)	Mauro
Exámenes	React	CU19, CU20, CU22, CU23 (banco de preguntas, simulacros, notas)	Otsubo
Contenido	React	CU32 (manuales)	Otsubo
Asistente IA	React	CU41, CU43 (IA local web y agentes n8n)	Leonel

El reparto de dueños sale del que ya tenían; es una sugerencia que pueden ajustar. Clases y horarios (CU12 a CU18), Notificaciones del usuario y Pagos del estudiante son solo móviles, así que no tienen micro-frontend.

Decisiones técnicas para que funcione con los dos frameworks

- **Module Federation:** Native Federation (`@angular-architects/native-federation`) en el shell Angular y el plugin de Module Federation para Vite en los remotos React. Cada remoto se compila y despliega por separado y expone un `remoteEntry`. El shell lo carga en tiempo de ejecución desde un manifiesto, sin recompilarse.

- **React dentro de Angular:** cada remoto React expone una función `mount(elemento, props)` y otra `unmount`. En el shell, un componente Angular genérico (`RemoteHost`) la invoca al navegar a la ruta. Los remotos Angular se cargan con `loadRemoteModule` normal.
- **Rutas:** el shell es dueño de la navegación de primer nivel (`/identidad/*`, `/escuela/*`, `/examenes/*`, etc.) y cada remoto maneja sus subrutas.
- **Sesión compartida:** solo el shell guarda el JWT y lo entrega a los remotos al montarlos. Los remotos no hacen login por su cuenta.
- **Comunicación entre remotos:** nunca se llaman entre sí. Si hace falta avisar algo (por ejemplo, "se creó un estudiante"), usan eventos del navegador (`CustomEvent`) o un pequeño bus que expone el shell.
- **Estilo común:** comparten variables CSS (colores, tipografía, espaciado) en un paquete de tokens, para que se vea como una sola aplicación pese a mezclar Angular y React.
- **API:** todos hablan con el gateway, y GraphQL sirve para las pantallas que juntan datos de varios servicios.

Con 4 piezas Angular (shell y 3 remotos) y 3 remotos React, el requisito de usar ambos frameworks queda cubierto.